

# [C 언어 2부] 같은 함수 안의 지역변수와 포인터는 메모리에서 어떻게 연결될까?

1. 포인터는 '연결선'이라는 암기 배열 편에서 `arr` 가 포인터 그 자체는 아니고, 대부분의 표현식 문맥에서 첫 원소의 주소로 붕괴(decay)할 뿐인 객체 라는 점을 봤다면, 이제는 그 다음 단계로 내려갈 차례다.

이번에는 "포인터 변수 자체..."

A

📅 2026년 3월 26일 ⌚ 9분 읽기 👤 Admin

#C #Pointer #Memory #Stack #Compiler #Local Variable

## 1. 포인터는 '연결선'이라는 암기

배열 편에서 `arr` 가 포인터 그 자체는 아니고, 대부분의 표현식 문맥에서 첫 원소의 주소로 붕괴(decay)할 뿐인 객체 라는 점을 봤다면, 이제는 그 다음 단계로 내려갈 차례다.

이번에는 "포인터 변수 자체가 메모리에서 어디에 놓이며, 지역변수끼리 서로를 참조할 때 실제로 무슨 일이 벌어지는가?" 를 봐야 한다.

보통의 설명에서는 흔히 "`p` 가 `x` 를 가리킨다" 라고만 말한다. 물론 틀린 말은 아니다. 그런데 이 표현만 머릿속에 남으면, 포인터를 눈에 보이지 않는 연결선이나 추상적인 링크처럼 상상하게 된다. 그렇게 외우면 `p = &x` , `p = &y` , `*p = 42` , `*pp = &y` , `**pp = 55` 같은 문장들이 복잡하다고 느껴지면서 머리가 뜨거워진다.

C에서 이 문제를 풀어내는 방법은 의외로 단순하다.

같은 함수에서 만든 지역변수들은 보통 같은 스택 프레임 안의 서로 다른 슬롯에 놓인다.  
그리고 포인터 변수는 그 슬롯들 중 하나의 주소값을 저장한다.

주소 값과 슬롯 배치는 이해를 위한 예시일 뿐이며, 실제 주소와 순서는 컴파일러, ABI, 최적화 옵션에 따라 달라질 수 있다. 그래도 "같은 프레임 안의 서로 다른 칸"이라는 개념 모델은 포인터를 이해하는 데 아주 정확하다고 볼 수 있다.

## 2. 같은 함수의 지역변수는 같은 스택 프레임 안에서 출발한다

우선 단순한 예시부터 보자.

```
void f(void) {  
    int x = 7;  
    int *p = &x;  
}
```

함수가 호출되면 보통 `f` 를 위한 스택 프레임이 만들어지고, 지역변수들은 그 프레임 내부의 슬롯을 차지한다. 개념적인 상태는 대략 이와 같다:

| 주소(예시) | 의미 | 값      |
|--------|----|--------|
| 0x1008 | x  | 7      |
| 0x1000 | p  | 0x1008 |

이 표에서 중요한 부분은

- `x` 는 **실제 정수 값**을 저장하는 칸이다.
- `p` 는 **주소값**을 저장하는 칸이다.

즉 "`p` 가 `x` 를 참조한다" 는 말은, 추상적으로는 그럴듯해 보이지만 실제 메모리 관점에서는 그냥 `p` 슬롯 안에 `x` 슬롯의 위치 숫자( `0x1008` )가 들어 있다 는 뜻인 것이다.

이 특성은 지역변수가 여러 개여도 유지된다.

```
void f(void) {  
    int a = 10;  
    int b = 20;  
    int *p = &a;  
    int *q = &b;  
}
```

이 상태는 다음처럼 볼 수 있다:

| 주소(예시) | 의미 | 값  |
|--------|----|----|
| 0x1018 | a  | 10 |
| 0x1014 | b  | 20 |
| 0x1008 | p  | &a |
| 0x1000 | q  | &b |

즉 a , b 는 값 저장 칸이고, p , q 는 주소 저장 칸으로 해석된다.

이미지 — 원본 이미지

3. p = &x , p = &y 에서 실제로 바뀌는 것은 누구인가?

포인터 이동을 다음 코드로 살펴보자:

```
void f(void) {
    int x = 7;
    int y = 99;
    int *p = &x;
    p = &y;
}
```

초기화 직후는 다음과 같을 것이다:

| 주소     | 의미 | 값  |
|--------|----|----|
| 0x1008 | x  | 7  |
| 0x1004 | y  | 99 |
|        |    |    |

| 주소     | 의미 | 값      |
|--------|----|--------|
| 0x1000 | p  | 0x1008 |

이 상태에서는 p 가 x 의 주소를 들고 있으므로, "p 가 x 를 가리킨다" 고 표현할 수 있다.

[이미지 — 원본 이미지](#)

그런데 다음 줄이 실행되면:

```
p = &y;
```

상태는 이렇게 바뀐다.

| 주소     | 의미 | 값      |
|--------|----|--------|
| 0x1008 | x  | 7      |
| 0x1004 | y  | 99     |
| 0x1000 | p  | 0x1004 |

이 상태 변화를 더 풀어서 살펴보면

- x 가 이동한 것이 아니다.
- y 가 이동한 것도 아니다.
- 바뀐 것은 오직 p 슬롯 내부의 숫자 뿐이다 라는 점을 알 수 있다.

우린 흔히 포인터를 "대상에 꽂힌 선"처럼 느끼는데, 실제 메모리에서는 그보다 훨씬 건조하게 관측된다. 즉, 메모리의 시점에서는 그냥 p 라는 변수 칸 안의 값이 0x1008 에서 0x1004 로 덮어써진 것이다.

[이미지 — 원본 이미지](#)

#### 4. 역참조는 포인터를 바꾸는 문장이 아니라, 포인터가 들고 있는 주소를 사용한다.

이번에는 쓰기 동작을 보자.

```
void f(void) {  
    int x = 7;  
    int y = 99;  
    int *p = &y;  
    *p = 42;  
}
```

`*p = 42;` 가 실행되기 직전 상태는 이렇다.

| 주소     | 의미 | 값      |
|--------|----|--------|
| 0x1008 | x  | 7      |
| 0x1004 | y  | 99     |
| 0x1000 | p  | 0x1004 |

이제 CPU 관점에서 `*p = 42;` 를 읽으면 다음 순서로 나타난다:

1. `p` 를 읽는다. 결과는 `0x1004` .
2. 그 숫자를 목적지 주소로 사용한다.
3. 그 위치에 `42` 를 저장한다.

결과는 다음과 같다:

| 주소     | 의미 | 값  |
|--------|----|----|
| 0x1008 | x  | 7  |
| 0x1004 | y  | 42 |

| 주소     | 의미 | 값      |
|--------|----|--------|
| 0x1000 | p  | 0x1004 |

즉 `*p = 42;` 는 `p` 를 바꾸는 문장이 아니라, `p` 가 가리키는 대상을 바꾸는 것이다. 단순하고 쉬운 개념이지만, 초반에는 헷갈리기 쉽다 (경험담).

[이미지 — 원본 이미지](#)

## 5. 이중 포인터부터는 '층'을 나눠서 해석하자!

이제 한 단계 더 내려가서,

```
void f(void) {
    int x = 1;
    int *p = &x;
    int **pp = &p;
}
```

여기서 지역변수는 3개로 나타난다:

- `x` : 실제 정수 값
- `p` : `x` 의 주소 저장
- `pp` : `p` 의 주소 저장

현 상태를 그리면 이렇게 나타날 것이다:

[이미지 — 원본 이미지](#)

이 그림을 굳이 설명해보면

- `pp` 는 `p` 를 가리킨다.
- `p` 는 `x` 를 가리킨다.
- 따라서 `**pp` 는 결국 `x` 에 접근한다.

실행 흐름으로 쓰면:

1. `pp` 를 읽는다. 결과는 `p` 의 주소다.
2. 그 주소로 가서 `p` 값을 읽는다. 결과는 `x` 의 주소다.
3. 그 주소로 가서 최종 `int` 값을 읽거나 쓴다.

헷갈릴 수 있으니 한번 층을 나누어서 살펴보자

- 1층: 실제 값 객체 ( `x` , `y` )
- 2층: 그 객체의 주소를 저장하는 포인터 객체 ( `p` , `q` )
- 3층: 포인터 객체 자체의 주소를 저장하는 상위 포인터 ( `pp` )

복잡할수록 더 쪼개서 살펴보면 상태 변화 추적이 덜 꼬인다.

## 6. `*pp = &y` 와 `**pp = 55` 는 완전히 다른 이유

이중 포인터를 처음보면 헷갈릴 수 있는 지점이다.

```
void f(void) {  
    int x = 10;  
    int y = 20;  
    int *p = &x;  
    int **pp = &p;  
  
    *pp = &y;  
}
```

이 문장은 `p` 자체를 수정한다. 즉, `pp` 가 가리키는 슬롯이 `p` 이므로, 그 안에 `&y` 를 써 넣는 것이다. 따라서 `p` 는 더 이상 `x` 를 가리키지 않고 `y` 를 가리키게 된다.

반대로 다음 코드는:

```
void f(void) {  
    int x = 10;  
    int *p = &x;
```

```
int **pp = &p;

**pp = 55;
}
```

이번에는 `p` 가 가리키는 **최종 대상 값**을 바꾼다. 즉 `x` 의 값이 `55` 로 바뀐다.

헛갈린다면, 차이를 그림으로 살펴보는 것이 이해에 훨씬 도움이 될 것이다:

[이미지 — 원본 이미지](#)

[이미지 — 원본 이미지](#)

즉, 정리하면:

- `*pp = &y;` 는 **포인터 재배선**이다.
- `**pp = 55;` 는 **최종 값 수정**이다.

이 간단하지만 중요한 차이를 놓치면, 이중 포인터는 거의 항상 "뭔가 되긴 되는데 왜 저 값이 바뀌는지 모르겠다" 는 상태에 빠질 수 있다.

## 7. 같은 함수 안에서는 안전하지만, 함수 밖으로 꺼내는 순간 수명이 깨진다

같은 함수 안에서 지역변수의 주소를 다른 지역 포인터에 저장하는 것 자체는 전혀 이상한 일이 아니다. 또한, 우린 흔하게 이 방법을 무의식적으로 사용하고 있다.

```
void f(void) {
    int x = 7;
    int *p = &x;
    printf("%d\n", *p);
}
```

여기서 의문이 생긴다. 함수 내부 변수들은 스택에 저장.. 근데 왜 안전할까?

- `x` 와 `p` 둘 다 `f` 의 지역변수다.



- `f` 가 끝나기 전까지 `x` 의 수명은 살아 있다.
- 따라서 `p` 가 들고 있는 주소도 그 시점에서는 유효하다.

문제는 이 주소를 함수 바깥으로 들고 나갈 때다.

```
int *f(void) {
    int x = 7;
    int *p = &x;
    return p;
}
```

함수 내부에서 `p` 는 분명 `x` 의 주소를 들고 있다. 하지만 `return p;` 후 `f` 가 끝나면, 그 스택 프레임은 더 이상 유효한 지역 객체의 집합이 아니다.

[이미지 — 원본 이미지](#)

[이미지 — 원본 이미지](#)

여기서 오해가 발생할 수 있다. 반환된 포인터 안에 `0x1008` 같은 숫자는 여전히 남아 있을 수 있다. 하지만 그 숫자가 더 이상 "유효한 `x` 객체의 주소" 라는 뜻을 보장하지는 않는다. 이것이 바로 **댕글링 포인터(dangling pointer)** 다.

즉, 같은 함수 안에서는 자연스럽게 안전한 참조가, 함수 바깥으로 나가는 순간 **수명(lifetime)** 문제로 바뀐다. 함수 내부 포인터 관련이 헷갈린다면 1부를 다시 읽어보는 것을 추천한다.

## 8. 포인터 복사는 '대상 복사'가 아닌 이유

마지막으로, 상태 변화가 섞여 나오는 코드를 살펴보자

```
void f(void) {
    int x = 3;
    int y = 8;
    int *p = &x;
    int *q = &y;

    *p = *q;
```

```
    q = p;  
    *q = 100;  
}
```

초기화가 끝난 시점의 변수 상태는 이렇다.

| 변수 | 값  |
|----|----|
| x  | 3  |
| y  | 8  |
| p  | &x |
| q  | &y |

이후의 변화는 그림과 같이 관찰된다.

이미지 — 원본 이미지

핵심만 다시 정리해서 살펴보자.

```
*p = *q;
```

오른쪽 `*q` 를 평가하면 `q` 안의 주소를 따라가 `y` 의 값 `8` 을 읽는다.  
그 다음 왼쪽 `*p` 의 목적지 주소를 찾으면 `x` 의 위치가 나오고, 거기에 `8` 을 저장한다.

결과:

- `x = 8`
- `y = 8`
- `p = &x`
- `q = &y`

이미지 — 원본 이미지

```
q = p;
```

이번에는 값 객체를 건드리지 않는다. 그냥 `p` 안의 주소값을 읽어서 `q` 슬롯에 복사한다.  
따라서 이제 `q` 도 `x` 를 가리킨다.

결과:

- `x = 8`
- `y = 8`
- `p = &x`
- `q = &x`

이미지 — 원본 이미지

```
*q = 100;
```

이제 `q` 가 들고 있는 주소는 `x` 의 주소이므로, `*q = 100;` 은 결국 `x` 를 바꾼다.

결과:

- `x = 100`
- `y = 8`
- `p = &x`
- `q = &x`

이미지 — 원본 이미지

즉 `q = p;` 는 **대상을 복사한 문장**이 아니라, **주소값을 복사한 문장**이다.

그 결과 `p`, `q` 는 같은 대상을 공유하게 되고, 그 뒤의 역참조 쓰기는 같은 객체를 건드리게 되는 것이다.

## 9. 단, 보통 실제 머신 코드에서는 더 최적화된다.

여기까지의 설명은 **개념 모델로서 살펴본 것이다**. 다만 실제 현실의 컴파일러는 최적화가 켜져 있으면 어떤 지역변수를 아예 스택 슬롯에 두지 않고 레지스터에만 둘 수도 있다.

예를 들어:

```
void f(void) {  
    int x = 7;  
    int y = x + 1;  
}
```

이런 코드는 실제 머신 코드에서 `x`, `y` 둘 다 레지스터만으로 처리될 가능성이 있다.

하지만 다음처럼 주소를 취하는 순간:

```
int x = 7;  
int *p = &x;
```

`x` 는 주소가 필요한 객체가 되므로, 실제 메모리 슬롯을 가질 가능성이 훨씬 커지게된다.  
그래서 포인터가 등장하면 "이 변수는 어디에 존재하며, 누구의 주소가 누구 안에 들어 있는가?" 도 같이 고려하여 살펴봐야한다.

같은 함수에서 생성된 지역변수끼리 서로 참조한다는 것은, 추상적으로는 "연결"처럼 보이지만 실제로는 훨씬 단순하게 해석 가능하다.

같은 스택 프레임 안의 어떤 슬롯에, 다른 슬롯의 주소 비트패턴이 저장되는 것.

그리고 상태 변화는 결국 두 부류로 나뉜다.

- **포인터 값 변화:** 누구를 가리키는지가 바뀐다.

예: `p = &y;` , `q = p;` , `*pp = &y;`

- **가리키는 대상 값 변화:** 실제 데이터가 바뀐다.

예: `*p = 10;` , `**pp = 20;` , `*q = *p;`